Understanding What You Have Achieved

Your Learning Journey: Week 3 Practical Session

Dear Student,

You have just completed a practical session that covered one of the most important concepts in programming: **making decisions**. You didn't just read about conditionals - you **used them** to build programs that think, choose, and respond intelligently.

This document will help you understand exactly what you have learned, why it matters, and how it connects to your future as a developer.

Read through this carefully. You might be surprised by how much you already know!

- **Your Achievement Map**

WEEK 3: CONTROL FLOW – CONDITIONALS

- ✅ Wrote if-else-if statements for multiple conditions
- ✅ Used nested if-else for validation and decision-making
- ✅ Built a menu-driven program with switch
- ✅ Simplified code with the ternary operator
- ✅ Analyzed complex boolean expressions
- ✅ Evaluated readability and efficiency of conditionals
- ✅ Debugged a faulty conditional
- ✅ Pushed Week 3 code to GitHub

YOU CAN NOW CONTROL THE FLOW OF YOUR PROGRAMS!

- **Learning Outcome 1: Design a decision structure that handles multiple conditions using nested if-else or switch appropriately**

What You Did:

In Tasks 1, 2, and 3, you created:

1. **PositiveNegativeZero.java** – Used if-else-if to check if a number is positive, negative, or zero.
2. **StudentGrade.java** – Used nested if-else to validate marks and assign a grade.
3. **MpesaMenu.java** – Used switch to handle a menu of options.

➢ **What You Learned:**

if-else-if: For Multiple Mutually Exclusive Conditions

java

```
if (condition1) {
   // Executes if condition1 is true
} else if (condition2) {
   // Executes if condition1 is false AND condition2 is true
} else if (condition3) {
   // Executes if condition1 and condition2 are false AND condition3 is true
} else {
   // Executes if all conditions are false
}
```

➢ **When to use:**

- When conditions are based on ranges (e.g., marks ≥ 70, marks ≥ 60)
- When conditions are mutually exclusive (only one can be true)
- When you need to check conditions in a specific order

Nested if-else: For Sequential Validation

java

```
if (outerCondition) {
   // Outer condition is true
   if (innerCondition) {
      // Both outer and inner conditions are true
```

```
    } else {
        // Outer is true, inner is false
    }
} else {
    // Outer condition is false
}
```

When to use:

- When you need to validate something before making a decision
- When decisions depend on multiple factors (e.g., valid marks → then assign grade)
- When you want to avoid checking invalid data

switch: For Multiple Exact Values

java

```
switch (variable) {
    case value1:
        // Executes if variable equals value1
        break;
    case value2:
        // Executes if variable equals value2
        break;
    default:
        // Executes if variable doesn't match any case
        break;
}
```

When to use:

- When you have multiple exact values (e.g., menu choices 1-7)
- When values are known and finite

- When readability is important (switch is cleaner than long if-else chains)

Comparison Table

Structure	Best For	Example
if-else-if	Ranges, ordered conditions	Grade calculation (≥70, ≥60, ≥50)
Nested if-else	Validation + decision	Check valid marks → assign grade
Switch	Exact values, menus	Menu options 1-4
Ternary	Simple true/false	Eligible or not eligible

Why This Matters:

Real-world systems make decisions constantly:

- **M-Pesa**: "Is the balance sufficient? If yes, approve. If no, decline."
- **Banking**: "Is the PIN correct? If yes, proceed. If no, retry."
- **E-commerce**: "Is the item in stock? If yes, add to cart. If no, show out of stock."

Choosing the right structure makes your code **readable**, **maintainable**, and **efficient**.

What You Can Now Do:

You can look at any decision-making problem and choose the most appropriate conditional structure (if-else-if, nested if-else, or switch) to solve it elegantly.

Learning Outcome 2: Analyze complex boolean expressions to determine all possible execution paths

What You Did:

In Task 5, you analyzed the following expression:

java

```
boolean result = (5 > 3) && (4 < 2) || (10 == 10);
```

You broke it down step by step:

- 5 > 3 → true
- 4 < 2 → false
- (true) && (false) → false (because AND requires both to be true)
- 10 == 10 → true
- false || true → true (because OR requires at least one to be true)

Final result: true

What You Learned:

Boolean Operators Review

Operator	Name	Rule	Example	Result
&&	AND	True only if BOTH are true	true && false	false
||	OR	True if AT LEAST ONE is true	true || false	true
!	NOT	Reverses the value	!true	false

Operator Precedence (for Booleans)

1. ! (NOT) – highest
2. && (AND)
3. || (OR) – lowest

Example:

java

```
boolean result = true || false && false;

// Step 1: false && false = false

// Step 2: true || false = true

// Result: true
```

Short-Circuit Evaluation

Java uses **short-circuit evaluation**:

- &&: If the left side is false, the right side is **NOT evaluated**.
- ||: If the left side is true, the right side is **NOT evaluated**.

java

int x = 5;

boolean result = (x > 10) && (x++ > 0);

// x > 10 is false, so x++ is NOT evaluated

System.out.println(x); *// Output: 5 (not 6)*

Why This Matters:

Boolean expressions are everywhere:

- **Login validation**: isUsernameCorrect && isPasswordCorrect
- **Loan approval**: age >= 18 && income >= 50000
- **Free shipping**: total >= 5000 || isPremiumMember

Understanding boolean logic allows you to:

- Write correct conditions
- Debug logical errors
- Optimize performance (using short-circuit)

What You Can Now Do:

You can analyze any boolean expression, predict its output, and explain the order of evaluation using precedence and short-circuit rules.

- **Learning Outcome 3: Evaluate the readability and efficiency of different conditional structures for the same logic**

What You Did:

In Task 6, you compared two versions of the same program:

Version A: if-else-if

java

```
if (day == 1) {
    System.out.println("Monday");
} else if (day == 2) {
    System.out.println("Tuesday");
} // ... continues to 7
```

Version B: switch

java

```
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    // ... continues to 7
}
```

What You Learned:

Readability Comparison

Aspect	if-else-if	switch
Visual clarity	Repetitive conditions	Clean cases
Lines of code	More lines	Fewer lines
Intent	"Check these conditions"	"Match this value"
Maintenance	Harder to add/remove	Easier to add/remove

Efficiency Comparison

Aspect	if-else-if	switch
Compilation	Sequential checks	Jump table (often)
Performance	O(n) – checks each condition	O(1) – direct jump
Best for	Ranges, complex conditions	Exact values

Note: For small numbers of conditions, the performance difference is negligible. Readability should be the primary concern.

When to Use Which

Scenario	Recommended Structure	Why
Menu with 5+ options	switch	Cleaner, easier to read
Grade calculation (ranges)	if-else-if	switch doesn't support ranges
2-3 conditions	if-else	Simpler
Complex boolean conditions	if-else-if	switch only works with exact values
String comparison (Java 7+)	switch	Supported since Java 7

Why This Matters:

Code is read **10 times more often than it is written**. Choosing the right structure:

- Makes your code easier to understand
- Reduces bugs during maintenance
- Helps team collaboration

- Shows professionalism

What You Can Now Do:

You can evaluate different conditional structures and choose the one that is most readable and appropriate for the given problem.

- **Learning Outcome 4: Debug a faulty conditional statement by tracing execution with various test inputs**

What You Did:

In Task 7, you examined this faulty code:

java

```
int age = 20;
if (age >= 18 || age <= 65) {
   System.out.println("You are of working age.");
} else {
   System.out.println("You are not of working age.");
}
```

You traced the code with different inputs:

- age = 20 → 20 >= 18 is true → true || ... → "Working age" (correct)
- age = 10 → 10 >= 18 is false, 10 <= 65 is true → false || true → "Working age" (WRONG!)
- age = 70 → 70 >= 18 is true → true || ... → "Working age" (WRONG!)

The bug: Used || instead of &&. The correct condition is age >= 18 && age <= 65.

What You Learned:

Debugging Process

1. **Read the code** – Understand what it's supposed to do.
2. **Trace with test inputs** – Use a table to track variable values.
3. **Identify the error** – Find where the logic deviates.
4. **Fix the error** – Change the condition.

5. **Re-test** – Verify the fix works for all inputs.

Common Conditional Errors

Error	Example	Fix
= instead of ==	if (x = 5)	if (x == 5)
&& vs ||	if (x > 0 || x < 10)	if (x > 0 && x < 10)
Missing braces	if (x > 0) System.out.println("A"); System.out.println("B");	Add { }
Off-by-one	if (x > 18) should be >= 18	Use correct comparison
Wrong order	if (x >= 60) before if (x >= 70)	Check highest first

Trace Table Example

Input	Condition	Result	Correct?
age = 10	10 >= 18 || 10 <= 65	true	❌ No
age = 20	20 >= 18 || 20 <= 65	true	✅ Yes
age = 70	70 >= 18 || 70 <= 65	true	❌ No

Why This Matters:

Debugging is 50% of a developer's job. Learning to trace code methodically:

- Saves hours of frustration
- Builds confidence

- Improves code quality
- Prepares you for complex systems

What You Can Now Do:

You can identify and fix logic errors in conditional statements by tracing execution with multiple test inputs.

Your Progress: Where You Started vs. Where You Are Now

Before Week 3	After Week 3
Didn't know how to make programs "think"	Can write if-else-if, nested if-else, and switch
Didn't understand boolean logic	Can analyze complex boolean expressions
Used only if-else for everything	Can choose the right structure for the problem
Didn't know how to debug logic errors	Can trace and fix faulty conditionals
Didn't understand switch	Can build menu-driven programs

➢ **How to Prove You've Achieved These Outcomes**

Here are the **artifacts** (proof) you now have:

1. **GitHub Repository:** https://github.com/YOUR-USERNAME/java-course
2. **Seven Java Files in** week3**:**
    - PositiveNegativeZero.java
    - StudentGrade.java
    - MpesaMenu.java
    - TernaryDemo.java

- BooleanAnalysis.java
- DayIfElse.java and DaySwitch.java
- FaultyConditional.java and FixedConditional.java

3. **Commit History:** Shows your progress
4. **Reflection Notes:** Your answers to questions

These artifacts are your portfolio pieces. Employers can see them!

➢ **How This Connects to Your Career**

Learning Outcome	Career Application
Design decision structures	Every application requires decision-making. You can now implement it.
Analyze boolean expressions	Interviews often test boolean logic. You can solve these problems.
Evaluate code readability	Senior developers care about clean code. You are learning this.
Debug conditional errors	Debugging is a daily task. You have the skills.
Git & GitHub	Every employer expects you to know Git. Your portfolio is growing.

➢ **Student Reflection: In Your Own Words**

Answer these questions in your notebook. Be honest – this helps you learn:

1. Which conditional structure did you find most useful? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?

4. In your own words, when would you use switch instead of if-else?
5. How will conditionals help you build real-world applications?
6. What is one thing you want to learn more about?

WEEK 3: Control Flow – Conditionals

Learning Outcomes Achieved:

✅ Design decision structures with if-else, switch

✅ Analyze complex boolean expressions

✅ Evaluate readability and efficiency of conditionals

✅ Debug faulty conditional statements

✅ Use Git and GitHub

➢ **What's Next?**

In **Week 4**, you will:

1. Learn about **loops** (while, do-while, for)
2. Write programs that **repeat tasks**
3. Build a **number guessing game** and **pattern generator**
4. Continue pushing your code to GitHub

But before you move on, make sure you can answer YES to these questions:

- □ I can write if-else-if statements for multiple conditions.
- □ I can use nested if-else for validation and decision-making.
- □ I can use switch statements for menu-driven programs.
- □ I can use the ternary operator for simple conditions.
- □ I can analyze complex boolean expressions.
- □ I can evaluate the readability and efficiency of different conditional structures.
- □ I can debug faulty conditional statements.

- □ I have pushed my Week 3 code to GitHub.
- □ I am excited to learn more!

If you answered YES to all, you are ready for Week 4!

➢ **Key Terms Glossary**

Term	Definition
Conditional	A statement that makes a decision based on a condition
if-else-if	A chain of conditions checked in order
Nested if-else	An if-else inside another if-else
switch	A multi-way branch based on exact values
Ternary Operator	A shorthand for simple if-else (? :)
Boolean Expression	An expression that evaluates to true or false
Short-Circuit	Skipping evaluation when the result is already known
Trace	Manually following code execution step by step
Fall-through	In switch, executing the next case without break
Debug	Finding and fixing errors in code

➢ **Final Words**

"First, solve the problem. Then, write the code." – John Johnson

You started this week not knowing how to make programs "think." You end this week with:

- **Seven working programs**
- **A growing GitHub portfolio**
- **A clear understanding of conditionals**
- **The ability to debug logic errors**

This is a critical skill. Every program you ever write will use conditionals. You now understand them.

Keep going!

END OF ACHIEVEMENT GUIDE FOR WEEK 3

"The best way to predict the future is to invent it." – Alan Kay

---

Instructor Note: Share this document with students at the end of Week 3. I encourage you to keep this document as a reference and to fill in the reflection questions. This helps consolidate your learning and gives you a tangible sense of accomplishment. Use the self-assessment checklist to identify your areas of weakness that need to be addressed before you go to week 4.